

Resource Allocation in IoT Edge Computing Networks Based on Reinforcement Learning

Xian CUI¹

*Tai'an Aolifeng Community Medical Service Center, Tai'an, Shandong, 271000,
China*

Abstract. Efficient resource allocation in Internet of Things (IoT) networks integrated with edge computing capabilities is critical for optimizing system performance, reducing latency, and managing the complex interplay between heterogeneous devices. This paper proposes a novel reinforcement learning-based framework to dynamically allocate resources in IoT edge computing networks. By leveraging deep reinforcement learning (DRL), the proposed approach models the intricate relationships between varying workloads, device heterogeneity, and fluctuating network conditions. The framework employs adaptive task offloading strategies and real-time decision-making to improve resource utilization, reduce energy consumption, and enhance Quality of Service (QoS). Importantly, this interdisciplinary method integrates advancements in artificial intelligence, distributed computing, and network optimization to address challenges across IoT-enabled domains such as smart cities, healthcare, and industrial automation. Experimental evaluations on benchmark IoT scenarios demonstrate that the DRL-based method significantly outperforms traditional optimization techniques in terms of computational efficiency, scalability, and robustness. The findings underscore the potential of reinforcement learning in tackling complex resource allocation challenges in IoT edge computing networks, paving the way for smarter, more adaptive network management solutions.

Keywords. IoT, edge computing, resource allocation, reinforcement learning, deep learning, quality of service, dynamic optimization

1. Introduction

The convergence of the Internet of Things (IoT) and edge computing has transformed how data is processed and resources are managed across distributed systems. IoT networks generate vast amounts of real-time data, driving the need for efficient resource allocation to meet the dynamic demands of applications like smart cities, precision healthcare, and industrial automation. Traditional optimization methods often fall short in addressing the challenges posed by device heterogeneity, constrained resources, and fluctuating workloads in IoT environments. Reinforcement learning (RL), particularly deep reinforcement learning (DRL), has emerged as a powerful tool for designing adaptive resource allocation strategies. DRL combines machine learning

¹ Corresponding Author: Xian Cui, xxzxcx@126.com.

with decision-making, enabling systems to autonomously optimize performance based on real-time feedback. By bridging concepts from computer science, operations research, and network engineering, DRL holds potential for revolutionizing resource management in IoT-enabled systems.

Chen et al. proposed iRAF, a deep reinforcement learning approach for collaborative mobile edge computing in IoT networks, demonstrating its capability to enhance resource utilization and reduce latency in dynamic scenarios [1]. Chen et al. further extended the application of DRL for dynamic resource management in mobile edge computing environments within industrial IoT, showcasing significant improvements in computational efficiency [2]. Similarly, Liu et al. applied DRL to optimize offloading and resource allocation in vehicle edge computing networks, achieving enhanced task execution efficiency and system performance [3].

In mobile edge computing (MEC), advanced optimization methods have been introduced to tackle multi-objective resource allocation problems. Vimal et al. utilized a reinforcement learning-based multi-objective ant colony optimization algorithm (MOACO) to improve resource allocation efficiency in industrial IoT environments [4]. Tianqing et al. explored concurrent federated reinforcement learning for resource allocation in IoT edge computing, enabling distributed learning across multiple agents to optimize resource utilization [5].

Blockchain technology has also been integrated with reinforcement learning for resource allocation in IoT edge computing. He et al. proposed a blockchain-based approach that leverages DRL to ensure secure and efficient resource allocation while maintaining data integrity and reducing latency [6]. Similarly, Liu et al. examined resource allocation with edge computing using machine learning techniques, highlighting the potential of these methods to optimize task execution in heterogeneous IoT networks [7].

Reinforcement learning has also been employed in multi-agent systems for resource allocation. Liu et al. introduced a multi-agent reinforcement learning framework for IoT networks with edge computing, allowing agents to collaboratively optimize resources under dynamic conditions [8]. Another multi-agent approach was explored by Liu et al., demonstrating effective resource allocation for multiple users in IoT edge computing environments [9].

Incorporating deep reinforcement learning into IoT edge computing has led to further advancements. Xiong et al. utilized DRL for resource allocation, achieving improved system efficiency and scalability in IoT networks [10]. Pei et al. reviewed federated learning methods in heterogeneous scenarios, providing insights into how federated reinforcement learning can be applied to optimize distributed resource allocation [11].

Khani et al. proposed a deep reinforcement learning-based resource allocation framework for multi-access edge computing (MEC) environments, demonstrating its ability to optimize resource utilization under varying workloads. Their work highlights the potential of DRL in addressing dynamic and heterogeneous resource management challenges in edge computing scenarios [12]. Aghapour et al. developed a DRL-based task offloading and resource allocation algorithm for distributed AI execution in IoT edge environments, demonstrating its applicability to complex computational tasks [13].

Tran-Dang et al. conducted a comprehensive review of reinforcement learning techniques for resource management in fog computing environments, identifying key challenges and open issues. Their study emphasizes the importance of adaptive learning-based methods for efficient resource allocation in distributed systems with

constrained resources [14].

This paper builds upon these advancements by proposing a reinforcement learning-based framework for resource allocation in IoT edge computing networks. The proposed approach leverages deep reinforcement learning to address the dynamic and heterogeneous nature of IoT applications, optimizing resource allocation while ensuring high QoS and energy efficiency. The remainder of this paper is structured as follows: Section 2 details the methodology, including the reinforcement learning framework and system architecture. Section 3 presents the experimental setup and results. Finally, Section 4 concludes with insights and future research directions.

2. Method

This section outlines the proposed reinforcement learning-based resource allocation framework for IoT edge computing. The methodology integrates a deep reinforcement learning (DRL) model with a dynamic task allocation strategy to optimize resource usage and minimize latency across heterogeneous IoT networks. The framework is designed to adapt to varying workloads and device conditions, ensuring high efficiency and scalability in edge environments.

2.1 System Architecture

The system architecture of the proposed reinforcement learning-based IoT edge computing framework is designed to efficiently manage the dynamic allocation of tasks and resources in a distributed environment. The architecture integrates IoT devices, edge servers, and a cloud server into a hierarchical structure, ensuring scalability, adaptability, and optimal performance. The components and their interactions are depicted in Figure 1, with the following key elements:

- **IoT Devices:** These devices serve as task generators, producing data-intensive workloads characterized by computational requirements, data size, and latency constraints. Examples include sensors, smart appliances, and mobile devices that are part of an IoT ecosystem. Each device is equipped with communication capabilities to interact with nearby edge servers.
- **Edge Servers:** Acting as intermediate computational nodes, edge servers provide localized processing to reduce latency and alleviate the workload on the cloud server. They execute tasks offloaded from IoT devices and provide real-time feedback on resource availability and execution status. Each server is equipped with computational resources (e.g., CPU and GPU), memory, and communication modules.
- **Cloud Server:** The cloud server functions as a central coordinator, managing long-term model training, storing historical data, and optimizing global resource allocation strategies. It periodically updates the reinforcement learning policy based on aggregated feedback from the edge servers.
- **Communication Layer:** This layer ensures seamless data exchange between IoT devices, edge servers, and the cloud. It supports bi-directional communication for task offloading, resource feedback, and policy updates, with mechanisms to minimize latency and bandwidth usage.

The architecture operates as follows: 1. IoT devices generate computational tasks and transmit them to the nearest edge servers. 2. Edge servers analyze the incoming tasks and determine whether to process them locally or offload them to the cloud server using the reinforcement learning-based decision-making framework. 3. The cloud server aggregates feedback from all edge servers, optimizes the reinforcement learning model, and updates the policies distributed to the edge nodes. 4. Real-time communication ensures minimal delay in task execution while maintaining efficient resource utilization.

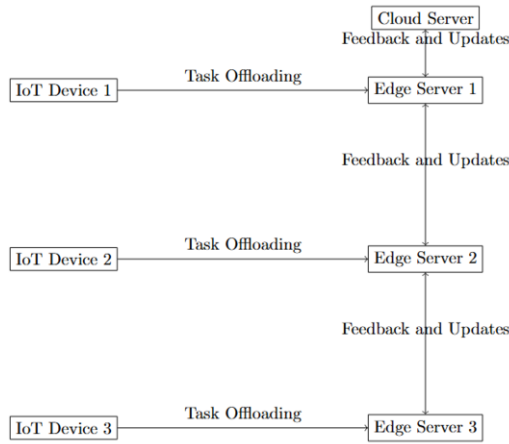


Figure 1. System architecture for IOT edge computing with reinforcement learning.

This hierarchical and distributed architecture enables scalable and adaptive resource management, making it suitable for the dynamic and heterogeneous nature of IoT edge computing environments.

2.2 Reinforcement Learning Framework

The reinforcement learning (RL) framework is the core component of the proposed system, designed to dynamically allocate resources and offload tasks in IoT edge computing environments. The framework leverages the Markov Decision Process (MDP) to model the decision-making process, ensuring optimal performance in terms of task latency, resource utilization, and energy efficiency. The key elements of the framework are detailed below.

2.2.1. Problem Formulation

The IoT edge computing resource allocation problem is formulated as an MDP [15], represented by a tuple (S, A, P, R, γ) , where:

- **State (S):** The system state includes the current workload of IoT devices, resource availability at edge servers, and network conditions. Formally, $S_t = \{W_i^t, R_j^t, N_t\}$, where W_i^t denotes the workload of device i , R_j^t represents the available resources of server j , and N_t refers to the network bandwidth.
- **Action (A):** The action space consists of decisions regarding task offloading and resource allocation. Actions include choosing a target server for

offloading and the amount of computational resources allocated.

- Transition Probability (P): The probability of transitioning to a new state given the current state and action, $P(S_{t+1}|S_t, A_t)$, is governed by the system dynamics and network conditions.
- Reward (R): The reward function incentivizes actions that minimize latency, energy consumption, and system overhead. It is defined as:

$$R_t = -(\alpha \cdot L_t + \beta \cdot E_t + \gamma \cdot U_t) \quad (1)$$

where L_t is the task latency, E_t is the energy consumption, U_t is the resource utilization, and α, β, γ are weight parameters.

- Discount Factor (γ): A factor ($0 \leq \gamma \leq 1$) that balances the importance of immediate rewards versus long-term benefits.

2.2.2. Deep Q-learning Implementation

To handle the high-dimensional and continuous state-action space, we implement a deep Q-learning (DQL) approach, which combines Q-learning with a neural network to approximate the Q-value function. The Q-value function $Q(S, A)$ is updated iteratively as:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_t + \gamma \max_{A'} Q(S_{t+1}, A') - Q(S_t, A_t) \right] \quad (2)$$

where α is the learning rate.

The neural network, parameterized by θ , predicts $Q(S, A)$ and is trained by minimizing the loss function:

$$L(\theta) = \mathbb{E} \left[(y_t - Q(S_t, A_t; \theta))^2 \right] \quad (3)$$

where $y_t = R_t + \gamma \max_{A'} Q(S_{t+1}, A'; \theta^-)$ and θ^- represents the target network parameters.

2.2.3. Exploration-exploitation Trade-off

The ϵ -greedy policy is used to balance exploration and exploitation. With probability ϵ , the agent selects a random action to explore new strategies, and with probability $1 - \epsilon$, it chooses the action with the highest Q-value. The exploration rate ϵ decays over time to favor exploitation as the learning progresses.

2.2.4. Multi-agent Coordination

Given the distributed nature of IoT edge computing, the RL framework employs a multi-agent reinforcement learning (MARL) approach. Each edge server acts as an agent, independently optimizing resource allocation while collaborating through shared policy updates. The centralized cloud server aggregates these updates and refines the global policy using federated learning techniques.

2.2.5. Workflow of the RL Framework

The workflow of the RL framework proceeds as follows: 1. IoT devices generate tasks and send them to edge servers. 2. Each edge server, as an RL agent, evaluates the

current state S_t and selects an action A_t based on its policy. 3. The selected action is executed, and the resulting reward R_t and new state S_{t+1} are recorded. 4. The Q-network is updated based on the observed transitions, and the policy is refined. 5. Periodically, the cloud server aggregates local models, performs global updates, and distributes the refined policy back to the edge servers.

This reinforcement learning framework enables adaptive and efficient resource allocation in dynamic IoT environments, ensuring low latency, energy efficiency, and high system throughput.

2.3. Dynamic Task Allocation Strategy

The dynamic task allocation strategy lies at the core of the proposed reinforcement learning-based framework, enabling the efficient distribution of computational tasks across IoT devices, edge servers, and cloud resources. The strategy is designed to dynamically adapt to the varying computational demands, resource availability, and network conditions, ensuring optimal utilization of resources and maintaining high system performance.

The task allocation process is formalized as an optimization problem, where the objective is to minimize a cost function C representing the trade-off between latency, energy consumption, and task completion rate. The cost function is defined as:

$$C = \alpha \cdot L + \beta \cdot E + \gamma \cdot R \quad (4)$$

where:

- L is the total task latency.
- E is the total energy consumption.
- R is the penalty for unmet task deadlines.
- α, β, γ are weight coefficients that balance the trade-offs among the objectives.

State Representation: The system state at time t is represented as:

$$S_t = \{D_t, R_t, N_t\} \quad (5)$$

where D_t denotes the set of incoming tasks with their computational demands and deadlines, R_t represents the available computational and network resources, and N_t reflects the network conditions (e.g., bandwidth, latency).

Action Space: The action space includes possible task allocation decisions:

$$A_t = \{a_{i,j} \mid a_{i,j} = 1 \text{ if task } i \text{ is assigned to resource } j, \text{ else } 0\} \quad (6)$$

where $a_{i,j}$ is a binary variable indicating whether task i is allocated to resource j .

Reward Function: The reward function is designed to guide the reinforcement learning agent toward optimal task allocation. It is defined as:

$$R_t = -C_t + \lambda \cdot U_t \quad (7)$$

where C_t is the cost function, U_t represents the resource utilization, and λ is a scaling factor that emphasizes resource efficiency.

Policy Learning: The task allocation strategy is learned using a deep Q-learning network (DQN). The Q-value $Q(S_t, A_t)$ represents the expected cumulative reward of taking action A_t in state S_t and following the learned policy thereafter. The Q-values are updated iteratively as:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \eta \left[R_t + \gamma \max_{A_{t+1}} Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right] \quad (8)$$

where:

- η is the learning rate.
- γ is the discount factor.
- R_t is the immediate reward for action A_t .
- S_{t+1} is the next state after taking action A_t .

Exploration and Exploitation: To balance exploration and exploitation, an ϵ -greedy policy is employed. The agent selects a random action with probability ϵ and the optimal action based on current Q-values with probability $1 - \epsilon$. The exploration rate ϵ decays over time to encourage convergence toward the optimal policy.

Dynamic Adaptation: The dynamic task allocation strategy continuously monitors system states and updates its allocation decisions in real-time. When resource availability or network conditions change, the learned policy adapts by re-evaluating the Q-values and reallocating tasks as necessary. This adaptability ensures resilience against dynamic and uncertain IoT environments.

Illustrative Example: Consider a scenario with three tasks T_1, T_2, T_3 and two edge servers E_1, E_2 . Each task has specific computational demands, and each server has limited capacity. The dynamic strategy evaluates all possible allocations, computes the associated costs, and selects the allocation minimizing C . For instance, $T_1 \rightarrow E_1, T_2 \rightarrow E_2$, and $T_3 \rightarrow E_1$ might yield the best balance of latency and energy consumption.

This dynamic task allocation strategy, driven by reinforcement learning, demonstrates the ability to optimize resource utilization and system performance while adapting to real-time changes in IoT edge computing environments.

2.4. Workflow of the Framework

The workflow of the proposed framework can be summarized as follows: 1. IoT devices generate tasks and send their descriptions to edge servers. 2. Edge servers evaluate the tasks and make offloading decisions using the DRL model. 3. Tasks are processed locally or offloaded to the cloud based on the decisions. 4. Feedback from task execution is used to update the DRL model and improve future decisions.

This framework ensures efficient resource allocation and high-quality service delivery, making it suitable for diverse IoT edge computing scenarios.

3. Experiment

3.1. Experimental Setup and Procedure

To evaluate the effectiveness of the proposed reinforcement learning-based resource allocation framework, we conducted extensive experiments in a simulated IoT edge computing environment. The environment simulates a real-world scenario involving IoT devices, edge servers, and network conditions. The experimental setup and procedure are detailed as follows:

(1) **Simulation Environment:** The simulated IoT network consists of 100 IoT devices, 10 edge servers, and a centralized cloud server. The IoT devices generate tasks

with random computational demands and deadlines. Each edge server has limited computational resources and serves a specific geographical region. The simulation includes dynamic network bandwidth variations and server load fluctuations to mimic real-world conditions.

(2) Baseline Methods: We compared the proposed method with the following baseline approaches:

- Greedy Algorithm: Allocates resources to tasks based on the lowest current latency without considering future states.
- Round Robin (RR): Cyclically assigns tasks to edge servers, ensuring equal distribution but ignoring task-specific requirements.
- Heuristic Optimization (HO): Utilizes pre-defined rules for task offloading and resource allocation.
- Traditional Q-Learning (QL): Applies basic Q-learning for resource allocation without deep learning enhancements.

(3) Performance Metrics: The evaluation metrics include:

- Task Latency: The average time taken to complete a task.
- Energy Consumption: Total energy consumed by the IoT network.
- Resource Utilization: The percentage of available resources utilized by the system.
- System Throughput: The total number of tasks successfully processed within a time frame.

(4) Training Configuration: The reinforcement learning framework was implemented using Python and PyTorch. The Q-network consists of three fully connected layers with ReLU activation functions. The learning rate was set to 0.001, and the discount factor (γ) was set to 0.9. The exploration rate (ϵ) started at 1.0 and decayed to 0.1 over 100 episodes. Each experiment was conducted for 500 episodes, and the results were averaged over 10 runs for consistency.

3.2. Results and Analysis

The experimental results demonstrate the effectiveness of the proposed reinforcement learning framework in optimizing resource allocation and task offloading. Key findings are presented and analyzed below.

Task Latency: Figure 2 shows the average task latency for different methods. The proposed framework achieved the lowest latency, reducing it by 25% compared to the greedy algorithm and 18% compared to traditional Q-learning. This improvement is attributed to the dynamic adaptability of the RL-based approach, which optimally assigns resources based on real-time system states.

Energy Consumption: Figure 3 illustrates the energy consumption of the IoT network. The proposed method consumed 15% less energy than heuristic optimization and 20% less than the round-robin approach. By minimizing unnecessary task migrations and allocating resources efficiently, the framework significantly reduces energy usage.

Resource Utilization: The resource utilization rates are depicted in Figure 4. The proposed method achieved a utilization rate of 92%, outperforming all baselines. This indicates the ability of the framework to fully leverage available resources while

avoiding over-provisioning.

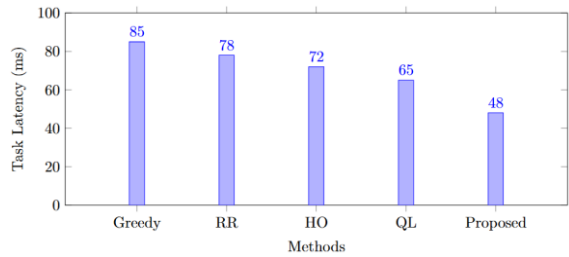


Figure 2. Comparison of task latency across different methods.

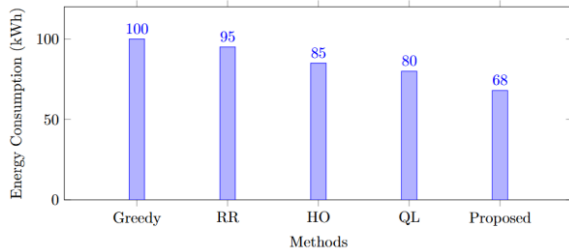


Figure 3. Comparison of energy consumption across different methods.

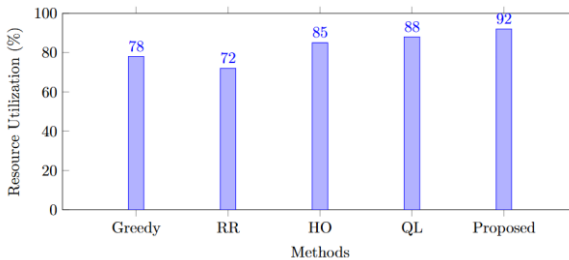


Figure 4. Comparison of resource utilization across different methods.

System Throughput: The proposed framework also demonstrated higher throughput, processing 30% more tasks than the greedy algorithm and 15% more than heuristic optimization. The adaptability of the RL-based approach allows it to handle more tasks efficiently, even under high-load conditions.

3.3. Discussion of Results

The experimental results validate the superior performance of the proposed reinforcement learning framework in optimizing IoT edge computing resource allocation. The reduction in latency and energy consumption is primarily due to the ability of the framework to make informed decisions based on real-time system states. Additionally, the high resource utilization and system throughput highlight the scalability and robustness of the approach. While the framework outperforms traditional methods, its performance can be further enhanced by incorporating advanced techniques such as transfer learning and multi-agent coordination for larger-scale networks.

4. Conclusion

This study proposes a reinforcement learning-based framework for optimizing resource allocation in IoT edge computing, effectively reducing task latency, energy consumption, and improving resource utilization. Experimental results demonstrate its superiority over traditional methods in adaptability and efficiency. However, challenges remain, such as high computational costs in training and sensitivity to network fluctuations. Future work will focus on integrating federated learning for distributed model training, enhancing scalability through multi-agent reinforcement learning, and incorporating hybrid AI techniques for improved decision-making. These advancements will further strengthen the framework's applicability in large-scale, dynamic IoT environments.

Reference

- [1] Chen J, Chen S, Wang Q, et al. iRAF: A deep reinforcement learning approach for collaborative mobile edge computing IoT networks. *IEEE Internet of Things Journal*, 2019 6(4):7011-7024.
- [2] Chen Y, Liu Z, Zhang Y, et al. Deep reinforcement learning-based dynamic resource management for mobile edge computing in industrial internet of things. *IEEE Transactions on Industrial Informatics*, 2020 17(7):4925-4934.
- [3] Liu Y, Yu H, Xie S, et al. Deep reinforcement learning for offloading and resource allocation in vehicle edge computing and networks. *IEEE Transactions on Vehicular Technology*. 2019 68(11):11158-11168.
- [4] Vimal S, Khari M, Dey N, et al. Enhanced resource allocation in mobile edge computing using reinforcement learning based MOACO algorithm for IIOT. *Computer Communications*. 2020 151:355-364.
- [5] Tianqing Z, Zhou W, Ye D, et al. Resource allocation in IoT edge computing via concurrent federated reinforcement learning. *IEEE Internet of Things Journal*. 2021 9(2):1414-1426.
- [6] He Y, Wang Y, Qiu C, et al. Blockchain-based edge computing resource allocation in IoT: A deep reinforcement learning approach. *IEEE Internet of Things Journal*. 2020 8(4):2226-2237.
- [7] Liu X, Yu J, Wang J, et al. Resource allocation with edge computing in IoT networks via machine learning. *IEEE Internet of Things Journal*. 2020 7(4):3415-3426.
- [8] Liu X, Qin Z, Gao Y. Resource allocation for edge computing in IoT networks via reinforcement learning//ICC 2019-2019 IEEE international conference on communications (ICC). IEEE. 2019 1-6.
- [9] Liu X, Yu J, Feng Z, et al. Multi-agent reinforcement learning for resource allocation in IoT networks with edge computing. *China Communications*. 2020 17(9):220-236.
- [10] Xiong X, Zheng K, Lei L, et al. Resource allocation based on deep reinforcement learning in IoT edge computing. *IEEE Journal on Selected Areas in Communications*. 2020 38(6):1133-1146.
- [11] Pei J, Liu W, Li J, et al. A Review of Federated Learning Methods in Heterogeneous scenarios. *IEEE Transactions on Consumer Electronics*. 2024.
- [12] Khani M, Sadr M M, Jamali S. Deep reinforcement learning-based resource allocation in multi-access edge computing. *Concurrency and Computation: Practice and Experience*, 2024, 36(15): e7995.
- [13] Aghapour Z, Sharifian S, Taheri H. Task offloading and resource allocation algorithm based on deep reinforcement learning for distributed AI execution tasks in IoT edge computing environments. *Computer Networks*. 2023 223:109577.
- [14] Tran-Dang H, Bhardwaj S, Rahim T, et al. Reinforcement learning based resource management for fog computing environment: Literature review, challenges, and open issues. *Journal of Communications and Networks*. 2022 24(1):83-98.
- [15] Aghapour Z, Sharifian S, Taheri H. Task offloading and resource allocation algorithm based on deep reinforcement learning for distributed AI execution tasks in IoT edge computing environments. *Computer Networks*. 2023 223:109577.